

AdvR 05 – Functions

Function Fundamentals

Three Components

Every R function has three parts:

- `formals()`: list of arguments
- `body()`: code inside the function
- `environment()`: where to look up names

```
f <- function(x, y) { x + y }  
formals(f)
```

```
#> $x  
#>  
#>  
#> $y
```

```
body(f)
```

```
#> {  
#>   x + y  
#> }
```

```
environment(f)
```

```
#> <environment: R_GlobalEnv>
```

Primitive Functions

Exception: **primitive functions** call C directly — all three components are NULL:

```
sum  
  
#> function (... , na.rm = FALSE) .Primitive("sum")
```

```
typeof(sum)
```

```
#> [1] "builtin"
```

```
formals(sum)
```

```
#> NULL
```

```
body(sum)
```

```
#> NULL
```

Two types: builtin (e.g., sum) and special (e.g., []). Only in the base package.

First-Class Functions and `do.call()`

Functions are **objects** — no special syntax needed:

```
f <- function(x) sin(1 / x ^ 2) # bind to name  
lapply(mtcars, function(x) length(unique(x))) # anonymous
```

R 4.1+ shorthand: `\(x) x + 1` is equivalent to `function(x) x + 1`.

Lists of functions:

```
funs <- list(half = \(x) x / 2, double = \(x) x * 2)  
funs$double(10)
```

```
#> [1] 20
```

`do.call()` calls a function with arguments from a list:

```
do.call(mean, list(1:10, na.rm = TRUE))
```

```
#> [1] 5.5
```

Exercises (Fundamentals)

- 1 Given a name like "mean", `match.fun()` finds a function. Given a function, can you find its name? Why doesn't that make sense in R?
- 2 Which of these is correct? Why?

```
function(x) 3()  
(function(x) 3)()
```

- 3 What function tells you if an object is a function? What about a primitive function?
- 4 What are the three important components of a function?

- ❶ No one-to-one mapping: a function can have **zero, one, or many** names. Multiple names can bind to the same function; some functions are anonymous.
- ❷ The second: `(function(x) 3)()`. Parentheses separate the anonymous function from the call `()`. The first creates a function with invalid body `3()`.
- ❸ `is.function()` and `is.primitive()`.
- ❹ `formals()` (arguments), `body()` (code), `environment()` (lookup context). Primitives have all three as `NULL`.

Lexical Scoping

Four Scoping Rules

R uses **lexical scoping**: looks up values based on where a function is **defined**, not where it is **called**.

1. Name masking — inner definitions shadow outer:

```
x <- 10  
g <- function() { x <- 20; x }  
g()
```

```
#> [1] 20
```

2. Functions vs. variables — R ignores non-functions in function-call position:

```
g09 <- function(x) x + 100  
g10 <- function() { g09 <- 10; g09(g09) }  
g10()
```

```
#> [1] 110
```

Four Scoping Rules (continued)

3. Fresh start — each call creates a new environment:

```
g11 <- function() {  
  if (!exists("a")) a <- 1  
  else a <- a + 1  
  a  
}  
c(g11(), g11())    # both 1
```

```
#> [1] 1 1
```

4. Dynamic lookup — values looked up at **run time**, not definition time:

```
g12 <- function() x + 1  
x <- 15; g12()
```

```
#> [1] 16
```

Use `codetools::findGlobals()` to detect external dependencies.

Exercises (Scoping)

- ❶ What does this return? How is each `c` interpreted?

```
c <- 10  
c(c = c)
```

- ❷ What are the four principles governing how R looks for values?

- ❸ Predict the result:

```
f <- function(x) {  
  f <- function(x) {  
    f <- function() x ^ 2  
    f() + 1  
  }  
  f(x) * 2  
}  
f(10)
```

- ① Returns `c(c = 10)`. First `c` = the `c()` function; second `c` = a quoted name; third `c` = the value 10.
- ② Name masking, functions vs. variables, fresh start, dynamic lookup.
- ③ $((10^2) + 1) * 2 = \mathbf{202}$. Innermost `f()` returns 100, middle adds 1 $\rightarrow$ 101, outer multiplies by 2.

Lazy Evaluation

Arguments Are Evaluated Only When Accessed

```
h01 <- function(x) 10  
h01(stop("This is an error!")) # no error!
```

```
#> [1] 10
```

Powered by **promises**: expression + environment + cached value. Evaluated at most **once**.

Default arguments can reference other arguments or variables defined later:

```
h04 <- function(x = 1, y = x * 2) c(x, y)  
h04()
```

```
#> [1] 1 2
```

Default arguments are evaluated **inside** the function; user-supplied arguments in the **calling** environment.

Missing Arguments

`missing()` detects whether argument was user-supplied:

```
h06 <- function(x = 10) {  
  list(missing(x), x)  
}  
str(h06())
```

```
#> List of 2  
#> $ : logi TRUE  
#> $ : num 10
```

```
str(h06(10))
```

```
#> List of 2  
#> $ : logi FALSE  
#> $ : num 10
```

The %||% Operator

The %||% operator provides a cleaner pattern for defaults:

```
`%||%` <- function(lhs, rhs) {  
  if (!is.null(lhs)) lhs else rhs  
}  
NULL %||% 42
```

```
#> [1] 42
```

Exercises (Lazy Evaluation)

- ❶ What important property of `&&` makes this work?

```
x_ok <- function(x) {  
  !is.null(x) && length(x) == 1 && x > 0  
}
```

What happens if you replace `&&` with `&`?

- ❷ What does this return? Which principle?

```
f2 <- function(x = z) { z <- 100; x }  
f2()
```

- ❶ **&& short-circuits**: if the LHS is `FALSE`, the RHS is never evaluated. This prevents `length(NULL) == 1` from running when `x` is `NULL`. With `&`, all conditions are evaluated (vectorized), causing errors or unexpected results for `NULL` or `length > 1` inputs.
- ❷ Returns **100**. Lazy evaluation: `x = z` is not evaluated until `x` is accessed, by which time `z <- 100` has already run inside the function.

... (Dot-Dot-Dot)

Passing Extra Arguments

... lets a function accept and pass on **any number** of extra arguments:

```
i02 <- function(x, ...) {  
  list(x = x, extras = list(...))  
}  
str(i02(1, a = 2, b = 3))
```

```
#> List of 2  
#> $ x      : num 1  
#> $ extras:List of 2  
#> ..$ a: num 2  
#> ..$ b: num 3  
#> ... [output truncated]
```

Two main uses:

- ① Pass arguments to inner functions (e.g., `na.rm` through `lapply`)
- ② Allow S3 methods to take different arguments

Downside: misspelled arguments silently absorbed:

```
sum(1, 2, NA, na_rm = TRUE) # na_rm, not na.rm!
```

```
#> [1] NA
```

Exercises (...)

1 Explain these results:

```
sum(1, 2, 3)
```

```
#> [1] 6
```

```
mean(1, 2, 3)
```

```
#> [1] 1
```

```
sum(1, 2, 3, na.omit = TRUE)
```

```
#> [1] 7
```

```
mean(1, 2, 3, na.omit = TRUE)
```

```
#> [1] 1
```

① `sum()` takes ... — all arguments are summed. `na.omit = TRUE` is coerced to 1, giving 7 instead of 6.

`mean()` has `x`, `trim`, `na.rm`, Arguments matched by position: `x = 1`, `trim = 2`, `na.rm = 3`. `na.omit` goes to ... (ignored). Only `x = 1` matters for the mean, so both calls return 1.

Exiting a Function

Implicit return: last evaluated expression:

```
j01 <- function(x) if (x < 10) 0 else 10  
j01(5)
```

```
#> [1] 0
```

Explicit return: via `return()`. Use for early exits only.

Invisible return — suppresses auto-printing:

```
j04 <- function() invisible(1)  
j04()           # nothing printed  
print(j04())    # force printing
```

```
#> [1] 1
```

`<-` returns invisibly, enabling chained assignment: `a <- b <- c <- 1`.

Exit Handlers with `on.exit()`

`on.exit()` runs cleanup code **regardless** of how the function exits:

```
j06 <- function(x) {  
  cat("Hello\n")  
  on.exit(cat("Goodbye!\n"), add = TRUE)  
  if (x) return(10) else stop("Error")  
}  
j06(TRUE)
```

```
#> Hello  
#> Goodbye!  
#> [1] 10
```

```
j06(FALSE)
```

```
#> Hello  
#> Error in j06(FALSE): Error
```

Restore global state (working directory, options, connections):

```
with_dir <- function(dir, code) {  
  old <- setwd(dir)  
  on.exit(setwd(old), add = TRUE)  
  force(code)  
}
```

Always use `add = TRUE` to avoid overwriting previous handlers.

- ❶ What does `load()` return? Why don't you normally see it?
- ❷ Write a function that opens a graphics device, runs code, and closes the device (always, even on error).

Solutions (Exiting)

- 1 `load()` invisibly returns a character vector of names of loaded objects. You don't see it because the return is invisible.

2

```
plot_pdf <- function(code) {  
  pdf("test.pdf")  
  on.exit(dev.off(), add = TRUE)  
  force(code)  
}
```

Function Forms

Four Calling Styles

Everything that exists is an object. Everything that happens is a function call. — John Chambers

Form	Example	Prefix equivalent
Prefix	<code>mean(x)</code>	<code>mean(x)</code>
Infix	<code>x + y</code>	<code>`+`(x, y)</code>
Replacement	<code>names(x) <- y</code>	<code>`names<`-(x, y)</code>
Special	<code>for (i in x) ...</code>	<code>`for`(i, x, ...)</code>

Everything can be rewritten in prefix form:

```
`+`(`1`, `*`(`2`, `3`)) # 1 + (2 * 3)
```

```
#> [1] 7
```

Argument Matching and Infix Functions

Arguments are matched by: (1) **exact name**, (2) **partial match**, (3) **position**.

```
# Confusing but valid:
x <- sample(replace = TRUE, 20, x = c(1:10, NA))
# Clear version:
x <- sample(c(1:10, NA), size = 20, replace = TRUE)
```

User-defined infix functions use %name%:

```
`%xor%` <- function(a, b) xor(a, b)
TRUE %xor% TRUE
```

```
#> [1] FALSE
```

```
FALSE %xor% TRUE
```

```
#> [1] TRUE
```

Replacement functions must return the modified object:

```
`random<-` <- function(x, value) {
  x[sample(length(x), 1)] <- value
  x
}
```

Exercises (Function Forms)

- 1 Rewrite in prefix form: $1 + 2 + 3$
- 2 Clarify these scrambled function calls:

```
x <- sample(replace = TRUE, 20, x = c(1:10, NA))  
y <- runif(min = 0, max = 1, 20)  
cor(m = "k", y = y, u = "p", x = x)
```

Solutions (Function Forms)

❶ ``+`(`+`(1, 2), 3)`

❷ Rewritten with explicit matching:

```
x <- sample(c(1:10, NA), size = 20, replace = TRUE)
y <- runif(20, min = 0, max = 1)
cor(x, y, use = "pairwise.complete.obs",
    method = "kendall")
```

Arguments matched: exact names first, then partial matching (`m` $\rightarrow$ `method`, `u` $\rightarrow$ `use`), then by position.